

AURA-PET

Documento de Requisitos de Producto (PDR) & Guía de Colaboración Git/GitHub

VERSIÓN

1.2.0

FECHA

Octubre 2026

EQUIPO

2+ Desarrolladores

CONTROL DE CÓDIGO

GitHub / Git-Flow

1. VISIÓN GENERAL DEL PRODUCTO

AuraPet es un asistente flotante de escritorio para Windows que combina un compañero interactivo SVG con físicas de resortes, automatización de terminal, panel de control desplegable y motor multi-proveedor de Inteligencia Artificial (Nube y Local Ollama).

2. STACK TECNOLÓGICO COMPLETO

- Core & Runtime:** Electron, Node.js, TypeScript.
- Frontend & UI:** React 18, Vite, Tailwind CSS.
- Animación Vectorial:** Framer Motion (físicas SVG e interacciones).
- Gestión de Estado:** Zustand (``petMoodStore``, ``uiStore``, ``settingsStore``).
- Comunicación External/CLI:** WebSocket Server (``ws``) local.
- Seguridad:** Electron ``safeStorage`` (Windows DPAPI) + ``contextBridge``.
- Testing & QA:** Vitest y Playwright.

3. REQUISITOS FUNCIONALES (RF) & NO FUNCIONALES (RNF)

- RF-01 (Ventana Flotante Transparente):** Ventana nativa frameless, transparente, siempre visible (always-on-top) y arrastrable.
- RF-02 (Mascota Vectorial SVG + Framer Motion):** Mascota reactiva con seguimiento de cursor, parpadeo aleatorio y físicas de resortes.
- RF-03 (Panel de Chat & Ajustes):** Despliegue de interfaz de conversación y configuración.
- RF-04 (Bandeja del Sistema):** Icono de System Tray con menú contextual.
- RF-05 (Receptor de Eventos WebSocket):** Servidor ``ws`` para escuchar alertas desde la CLI.
- RNF-01 / RNF-02 (Rendimiento y Seguridad):** Uso de CPU < 1%, RAM ≤ 120 MB y aislamiento de contexto (``contextIsolation: true``).

4. ARQUITECTURA DETALLADA DEL SISTEMA (CLEAN ARCHITECTURE)

AuraPet adopta un modelo estricto de desacoplamiento para garantizar la seguridad de Electron y facilitar el desarrollo simultáneo entre varios programadores sin colisiones de código.

4.1 Aislamiento de Procesos (Main vs Renderer):

- **Proceso Principal (Main - Node.js):** Responsable del ciclo de vida del sistema operativo, creación de ventanas transparentes, almacenamiento seguro mediante Windows DPAPI (`safeStorage`), persistencia en disco y servidor WebSocket. Ningún código de UI se ejecuta en Main.
- **Capa de Puente (Preload - Context Bridge):** Actúa como un *firewall* estricto. Expone únicamente una API tipada y congelada (`window.aurapetAPI`) hacia el renderer mediante `contextBridge.exposeInMainWorld()`. Desactiva explícitamente `nodeIntegration`.
- **Proceso de Renderizado (Renderer - React):** Responsable exclusivo de la interfaz gráfica, animaciones vectoriales SVG y gestión de estado visual con Zustand. No tiene acceso directo al sistema de archivos ni a Node.js.

4.2 Estructura Detallada de Carpetas:

```
src/ ├── main/ # PROCESO PRINCIPAL (Node.js) | ├── core/ # Capa de Dominio / Casos de Uso |
```

4.3 FLUJO DE DATOS E INTERACCIÓN IPC

Toda la comunicación entre la mascota y los servicios locales/nube sigue un flujo uni-direccional claro:



5. MATRIZ DE CONTROL DE CALIDAD Y RIESGOS

ID Req.	Componente	Caso de Prueba	Criterio de Aceptación
RF-01	`main/WindowManager`	`TC-WIN-001`	Ventana transparente frameless sin bordes ni destellos.
RF-02	`renderer/PetAvatar`	`TC-PET-001`	Respuesta de pupilas al cursor < 16ms a 60fps sin tirones.
RF-05	`main/sockets/WebSocketServer`	`TC-SOC-001`	Alertas por `ws://localhost:9001` cambian estado en < 50ms.
RF-07	`main/ai-providers/OllamaAdapter`	`TC-AI-001`	Intercepción de errores si el servicio de Ollama no responde.

6. MATRIZ DE RIESGOS

ID	Riesgo Identificado	Impacto	Estrategia de Mitigación
R-01	Carga de CPU por seguimiento del mouse.	Medio	Throttling con `requestAnimationFrame`.
R-02	Bloqueo de UI durante streaming de IA.	Alto	Procesamiento asíncrono no bloqueante en Main.
R-03	Conflictos en código por trabajo paralelo.	Crítico	Flujo estricto Feature-Branch con Pull Requests y Rebase.

7. GUÍA DE COLABORACIÓN EN EQUIPO (GITHUB WORKFLOWS)

Para trabajar de forma fluida entre 2 o más desarrolladores sin sobrescribir código ni generar conflictos en la rama principal (`main`), el equipo seguirá la siguiente estrategia:

- **Regla de Rama Principal (`main`):** La rama `main` debe ser sagrada y siempre estable/compilable. Se protegerá en GitHub impidiendo **direct pushes**.
- **Estrategia Feature-Branch:** Cada desarrollador trabajará en una rama individual para cada tarea (ej. `feature/pet-animations`, `feature/ollama-adapter`, `fix/socket-reconnect`).
- **Aprobación mediante Pull Request (PR):** Todo cambio requiere revisión de código (**Code Review**) por al menos 1 compañero antes de integrarse.

FLUJO PASO A PASO PARA UN INTEGRANTE DEL EQUIPO

```
# 1. Clonar el repositorio por primera vez git clone https://github.com/TU-ORGANIZACION/aura
```

GESTIÓN DE CONFLICTOS ENTRE DESARROLLADORES

Cuando 2 desarrolladores modifican el mismo archivo al mismo tiempo, sigue este procedimiento con `git rebase` para mantener un historial limpio:

```
# Mientras estás en tu rama de trabajo git fetch origin git rebase origin/main # Si Git repo
```

8. BUENAS PRÁCTICAS Y REGLAS DEL EQUIPO EN GITHUB

- **Conventional Commits:** Usar prefijos como ``feat:`` (nueva funcionalidad), ``fix:`` (corrección de bug), ``refactor:`` (mejora de código sin cambiar funcionalidad), ``docs:`` (documentación).
- **Integración Continua (CI):** Antes de aprobar un PR, GitHub Actions ejecutará automáticamente ``npm run test`` y ``npm run build``. Si las pruebas fallan, el PR no se podrá fusionar.
- **Variables de Entorno Locales:** NUNCA subir claves de API o archivos ``.env`` al repositorio. Se debe incluir un archivo de plantilla ``.env.example``.

9. PLANIFICACIÓN ÁGIL POR SPRINTS (EQUIPO DE 2+)

- **Sprint 1 (Fundaciones & Arquitectura):**
 - Dev A: Configuración de Electron + Clean Architecture, ``WindowManager`` y IPC Handlers.
 - Dev B: Configuración de React + Vite + Tailwind CSS y componentes de ``PetAvatar`` SVG.
- **Sprint 2 (UI & Físicas & Sockets):**
 - Dev A: Implementación del servidor de WebSockets y eventos de CLI local.
 - Dev B: Animaciones con Framer Motion, hooks de seguimiento de cursor y Zustand stores.
- **Sprint 3 (Integración de IA & Seguridad):**
 - Dev A: Integración de ``safeStorage`` (DPAPI) y adaptadores de OpenAI / Anthropic.
 - Dev B: Adaptador para Ollama/LM Studio local y panel de configuración UI.

10. CHEAT SHEET DE COMANDOS DE DESARROLLO Y GIT

```
# Comandos de Desarrollo del Proyecto npm run dev # Ejecuta la App con Hot Module Replacemen
```